



centre adscrit a:



UNIVERSITAT DE
BARCELONA

Proyecto final

MalScan

Gisela Esteve, Emma Rosendo y Nuria Salas

Profesor: David Berga

Disseny i Anàlisi d'Algorismes Avançats

Curs 2025-2026

1. Propuesta de nuevas funcionalidades

1.1 Funcionalidades previas

Desde el inicio del proyecto, el objetivo principal fue desarrollar una aplicación capaz de simular el funcionamiento básico de un sistema de detección de malware, aplicando conceptos de programación, estructuras de datos y análisis de archivos. Para ello, se definieron una serie de funcionalidades iniciales que permitieran construir una herramienta educativa, intuitiva y extensible, capaz de analizar archivos y clasificarlos según su nivel de riesgo.

Se buscó que el proyecto no solamente realizara un escaneo simple, sino que también incorporara características propias de soluciones reales de ciberseguridad, como el análisis mediante firmas, el recorrido recursivo de directorios, la generación de informes y la posibilidad de ampliar la base de detección añadiendo nuevas firmas maliciosas.

Las funcionalidades planteadas inicialmente fueron las siguientes:

1. **Carga de base de firmas de malware** El sistema debía ser capaz de cargar automáticamente una base de datos con firmas maliciosas previamente definidas, incluyendo hashes y patrones sospechosos.
2. **Estructura de directorios simulada** Se implementó una estructura de carpetas y archivos simulada para recrear un entorno realista de análisis de sistema.
3. **Recorrido recursivo de carpetas** La aplicación debía recorrer automáticamente todas las carpetas y subcarpetas para localizar y analizar cada archivo existente.
4. **Análisis por hash o patrones de contenido** Cada archivo sería analizado utilizando técnicas básicas de detección, comparando hashes SHA-256 y buscando patrones sospechosos dentro del contenido.
5. **Detección de coincidencias con firmas** El sistema debía identificar cuándo un archivo coincidía con alguna firma registrada en la base de datos.
6. **Clasificación: limpio / sospechoso / malicioso** Tras el análisis, los archivos serían clasificados según el nivel de riesgo detectado, facilitando la interpretación de resultados.
7. **Escaneo rápido vs. profundo** Se propuso incorporar distintos modos de análisis, permitiendo realizar escaneos rápidos o análisis más exhaustivos.
8. **Informe con resultados del análisis** La aplicación debía generar un informe final mostrando los resultados obtenidos durante el escaneo.
9. **Estadísticas de archivos analizados** También se planteó incluir estadísticas generales, como número de archivos analizados, cantidad de detecciones y porcentajes de riesgo.
10. **Añadir nuevas firmas al sistema** Finalmente, se diseñó el sistema con la intención de permitir la incorporación de nuevas firmas de malware de forma sencilla y escalable.

1.2 Funcionalidades actuales de MalScan

Aunque inicialmente se plantearon una serie de funcionalidades básicas orientadas al análisis de malware, durante el desarrollo del proyecto se decidió ampliar considerablemente las capacidades del sistema. El objetivo fue mejorar la experiencia de usuario, hacer la aplicación más intuitiva y acercar su funcionamiento al de una herramienta real de ciberseguridad.

A medida que avanzó la implementación, se añadieron nuevas características enfocadas tanto en la usabilidad como en la escalabilidad del proyecto. De este modo, MalScan pasó de ser un simple escáner basado en firmas a convertirse en una aplicación mucho más completa, visual y flexible.

Las funcionalidades actuales implementadas en MalScan son las siguientes:

1. **Interfaz gráfica en Tkinter** MalScan dispone de una interfaz gráfica desarrollada en Tkinter que permite interactuar con el sistema de forma visual, intuitiva y sencilla, facilitando el uso de la aplicación sin necesidad de utilizar comandos por terminal.
2. **Añadir archivos reales** El usuario puede seleccionar y añadir archivos reales desde su propio ordenador mediante un explorador de archivos integrado dentro de la interfaz.
3. **Estructura de directorios simulada** Los archivos añadidos se almacenan dentro de una estructura de árbol que simula el comportamiento de un sistema de directorios real.
4. **Escaneo rápido** El sistema permite realizar un escaneo rápido basado en la comparación de hashes SHA-256 con una base de firmas conocidas.
5. **Escaneo profundo** MalScan puede ejecutar un escaneo profundo que, además de verificar hashes, analiza el contenido interno de los archivos buscando patrones potencialmente maliciosos.
6. **Detección de amenazas** El programa detecta archivos sospechosos o maliciosos mediante coincidencias de hashes y patrones registrados en la base de firmas.
7. **Clasificación de archivos** Los archivos analizados se clasifican automáticamente según el nivel de riesgo detectado:
 - limpio,
 - sospechoso,
 - malicioso.
8. **Visualización de resultados** Los resultados del análisis se muestran en una tabla visual que incluye:
 - nombre del archivo,
 - nivel de riesgo,
 - firma detectada,
 - descripción de la amenaza.
9. **Visualización de hashes** El sistema permite consultar todos los hashes almacenados dentro de la base de datos de firmas maliciosas.

10. **Registro automático de hashes** Cuando se detecta un archivo malicioso cuyo hash todavía no existe en la base de datos, el sistema puede registrarlo automáticamente para futuras detecciones.
 11. **Visualización de archivos añadidos** MalScan permite visualizar todos los archivos cargados por el usuario junto con información relevante como:
 - tamaño del archivo,
 - hash SHA-256 generado.
 12. **Añadir firmas dinámicamente** El sistema permite incorporar nuevas firmas maliciosas de forma dinámica, sin necesidad de modificar el núcleo principal del programa.
 13. **Exportación de informes** MalScan permite exportar informes completos con los resultados obtenidos durante el análisis, incluyendo:
 - archivos analizados,
 - amenazas detectadas,
 - nivel de riesgo,
 - hashes generados,
 - estadísticas del escaneo.
 14. **Uso de recursividad** El recorrido de carpetas y subcarpetas se realiza mediante algoritmos recursivos, permitiendo analizar estructuras complejas de directorios.
 15. **Uso de POO y polimorfismo** El proyecto ha sido desarrollado utilizando programación orientada a objetos (POO), aplicando además polimorfismo para implementar diferentes estrategias de escaneo dentro del sistema.
-

1.3 Posibles funcionalidades futuras

Aunque MalScan actualmente dispone de múltiples funcionalidades orientadas al análisis y detección de malware, durante el desarrollo del proyecto también se plantearon diversas mejoras futuras que permitirían ampliar todavía más sus capacidades. Estas ideas surgen con el objetivo de hacer el sistema más eficiente, automatizado, escalable y cercano al funcionamiento de herramientas profesionales utilizadas en entornos reales de ciberseguridad.

Muchas de estas funcionalidades futuras están enfocadas en mejorar el rendimiento del análisis, automatizar procesos de detección y ampliar la compatibilidad con distintos tipos de amenazas y archivos. Además, permitirían que MalScan evolucionara hacia una plataforma más inteligente y preparada para escenarios más complejos.

Las posibles funcionalidades futuras planteadas son las siguientes:

1. **Carga de firmas desde archivos externos** Se plantea permitir la importación de firmas maliciosas desde archivos externos en formatos como **.json** o **.csv**, así como desde bases de datos remotas, facilitando la actualización y ampliación del sistema.
2. **Sistema de cuarentena** Otra mejora prevista consiste en implementar un sistema de cuarentena capaz de mover automáticamente los archivos maliciosos detectados a una carpeta aislada para evitar su ejecución accidental.

3. **Monitorización en tiempo real** Se podría incorporar un sistema de monitorización continua que analice automáticamente nuevos archivos añadidos a carpetas específicas del sistema.
4. **Optimización mediante Trie o Aho-Corasick** Con el objetivo de mejorar el rendimiento del análisis profundo, se plantea utilizar estructuras y algoritmos más eficientes, como Trie o Aho-Corasick, para acelerar la búsqueda de patrones maliciosos dentro de los archivos.
5. **Persistencia de datos** También se contempla implementar mecanismos de persistencia que permitan guardar automáticamente hashes, firmas y resultados para mantener la información entre distintas ejecuciones del programa.
6. **Estadísticas avanzadas** En futuras versiones podrían añadirse gráficos y estadísticas más avanzadas relacionadas con:
 - amenazas detectadas,
 - tipos de malware,
 - tiempo de escaneo,
 - nivel de riesgo.

Esto permitiría visualizar de forma más clara el comportamiento del sistema y los resultados obtenidos.

7. **Actualización automática de firmas** Otra funcionalidad futura sería la posibilidad de descargar automáticamente nuevas firmas maliciosas desde un servidor remoto, manteniendo la base de datos constantemente actualizada.
 8. **Compatibilidad con más tipos de archivos** Actualmente el sistema puede analizar distintos archivos básicos, pero en futuras versiones se pretende ampliar la compatibilidad para analizar:
 - scripts,
 - ejecutables,
 - documentos,
 - archivos comprimidos.
 9. **Machine Learning básico** Finalmente, se plantea incorporar técnicas básicas de aprendizaje automático capaces de detectar comportamientos sospechosos incluso cuando no exista una coincidencia exacta con firmas conocidas, permitiendo así mejorar la detección de amenazas desconocidas o variantes nuevas de malware.
-

2. Análisis de tiempos promedios

2.1 app.py

Este archivo contiene la interfaz gráfica y coordina la interacción del usuario.

crear_sistema_simulado()

Operación

- Crea la carpeta raíz.

Tiempo promedio

$O(1)$

crear_base_firmas()

Operación

- Carga las firmas iniciales.
- Actualmente se cargan 12 firmas.

Cálculo

- Si el número de firmas es fijo:

$O(1)$

- Si se generaliza a s firmas:

$O(s)$

Memoria

$O(s)$

cargar_archivos_guardados()

Operación

- Carga archivos persistentes desde JSON.

Cálculo

- Si hay f archivos guardados y cada uno tiene tamaño medio m :

$$T(f, m) = f \cdot m$$

- Resultado:

$$O(f \cdot m)$$

Tiempo promedio

$$O(f \cdot m)$$

Memoria

$$O(f \cdot m)$$

- porque reconstruye los contenidos en memoria.

guardar_archivos()

Operación

- Guarda en JSON los archivos añadidos.

Cálculo

- Debe recorrer todos los archivos y escribir su contenido:

$$T(f, m) = O(f \cdot m)$$

Tiempo promedio

$$O(f \cdot m)$$

Memoria

$$O(f \cdot m)$$

- por la estructura temporal de datos que se serializa.

obtener_o_crear_carpeta_usuario()

Operación

- Busca la carpeta **archivos_usuario**.

Cálculo

- Si hay **c** carpetas:

$O(c)$

- Normalmente **c** es pequeño, pero formalmente:

$O(c)$

crear_interfaz()

Operación

- Crea todos los elementos gráficos.

Cálculo

- El número de widgets es fijo.

$O(1)$

Memoria

$O(1)$

- respecto al número de archivos.

actualizar_tarjetas()

Operación

- Actualiza las estadísticas visuales.

Cálculo

- Si hay resultados:

$$T(r) = r + r + r$$

- porque cuenta limpios, sospechosos y maliciosos.

$$T(r) = O(r)$$

- Si no hay resultados, cuenta archivos:

$$O(f)$$

Tiempo promedio

$$O(r)$$

- o equivalentemente:

$$O(f)$$

obtener_archivos_anadidos()

Operación

- Recorre la estructura para obtener archivos.

Cálculo

- Si hay **c** carpetas y **f** archivos:

$$O(c + f)$$

- Como $c + f \leq n$:

$$O(n)$$

Tiempo promedio

$O(n)$

anadir_archivo()

Operación

- Permite seleccionar un archivo, leerlo, añadirlo y guardarlo.

Cálculo

1. Leer archivo:

$O(m)$

2. Buscar/crear carpeta:

$O(c)$

3. Añadir a lista:

$O(1)$

4. Guardar archivos persistentes:

$O(f \cdot m)$

- Total:

$T(f, m, c) = O(m) + O(c) + O(1) + O(f \cdot m)$

- Dominante:

$O(f \cdot m)$

Tiempo promedio

$$O(f \cdot m)$$

- porque guardar la persistencia reescribe todos los archivos.

eliminar_archivo()

Operación

- Elimina un archivo seleccionado.

Cálculo

1. Obtener archivos:

$$O(n)$$

2. Reconstruir lista sin el archivo:

$$O(f)$$

3. Guardar persistencia:

$$O(f \cdot m)$$

- Total:

$$O(n + f + f \cdot m)$$

- Dominante:

$$O(n + f \cdot m)$$

Tiempo promedio

$$O(n + f \cdot m)$$

ejecutar_escaneo()

Operación

- Ejecuta el escaneo y actualiza la tabla.

Cálculo

- Primero obtiene archivos:

$$O(n)$$

- Luego ejecuta el motor.
- Escaneo rápido:

$$O(n + f \cdot m)$$

- Escaneo profundo:

$$O(n + f \cdot p \cdot m)$$

- Después recorre resultados para mostrarlos:

$$O(r)$$

- También puede registrar hashes detectados:

$$O(f \cdot m)$$

- en el peor caso si debe calcular hashes de detectados.

Total promedio con escaneo rápido

$$O(n + f \cdot m + r)$$

- Como $r \approx f$:

$$O(n + f \cdot m)$$

Total promedio con escaneo profundo

$$O(n + f \cdot p \cdot m + r)$$

- Resultado:

$$O(n + f \cdot p \cdot m)$$

registrar_hash_detectado()

Operación

- Cuando un archivo es sospechoso o malicioso, guarda su hash si no existe.

Cálculo

1. Buscar archivo por nombre:

$$O(f)$$

2. Calcular hash:

$$O(m)$$

3. Comprobar diccionario de hashes:

$$O(1)$$

- Total:

$$O(f + m)$$

Tiempo promedio

$$O(f + m)$$

mostrar_archivos()

Operación

- Muestra archivos añadidos y calcula su hash.

Cálculo

- Para cada archivo:

$\text{calcular_hash} = O(m)$

- Para f archivos:

$O(f \cdot m)$

Tiempo promedio

$O(f \cdot m)$

mostrar_hashes()

Operación

- Muestra todos los hashes registrados.

Cálculo

- Si hay h hashes:

$O(h)$

Tiempo promedio

$O(h)$

mostrar_firmas()

Operación

- Muestra patrones y hashes.

Cálculo

- Debe recorrer:
 - **p** patrones,
 - **h** hashes.

$$T(p, h) = O(p + h)$$

Tiempo promedio

$$O(p + h)$$

exportar_informe()

Operación

- Decide si exporta TXT, CSV o HTML.
- La parte de selección es constante:

$$O(1)$$

- El coste real está en escribir el informe.

exportar_informe_html()

Operación

- Genera informe visual HTML.

Cálculo

1. Recorre resultados:

$$O(r)$$

2. Recorre hashes:

$$O(h)$$

3. Escribe contenido HTML:

$O(r + h)$

- Total:

$O(r + h)$

- Como $r \approx f$:

$O(f + h)$

Tiempo promedio

$O(f + h)$

limpiar_tabla()

Operación

- Borra filas visuales.

Cálculo

- Si hay r filas:

$O(r)$

Tiempo promedio

$O(r)$

limpiar_resultados()

Operación

- Limpia tabla, resultados y tarjetas.

Cálculo

$$O(r) + O(1) + O(f)$$

- Resultado:

$$O(r + f)$$

- Como normalmente $r \approx f$:

$$O(f)$$

borrar_archivos_guardados()

Operación

- Borra todos los archivos persistentes.

Cálculo

1. Reinicia estructura:

$$O(1)$$

2. Borra JSON:

$$O(1)$$

3. Limpia tabla:

$$O(r)$$

4. Actualiza tarjetas:

$$O(f)$$

- Total:

$$O(r + f)$$

Tiempo promedio

$$O(f)$$

2.2 estrategias.py

Este archivo implementa el polimorfismo del sistema.

EstrategiaEscaneo

Es una clase abstracta, por tanto no ejecuta análisis directo.

Tiempo promedio

$$O(1)$$

EscaneoRapido.escanear_archivo()

Operaciones

1. Calcular hash.
2. Buscar hash en diccionario.
3. Crear resultado.

Cálculo

$$T(m, h) = \text{calcular_hash} + \text{buscar_hash} + \text{crear_resultado}$$

- Sustituyendo:

$$T(m, h) = O(m) + O(1) + O(1)$$

- Por tanto:

$$T(m, h) = O(m)$$

Tiempo promedio

$$O(m)$$

Memoria

$$O(m)$$

- por la conversión temporal a bytes al calcular el hash.

EscaneoProfundo.escanear_archivo()

Operaciones

1. Calcula hash.
2. Busca hash.
3. Si no hay coincidencia, busca patrones.
4. Crea resultado.

Cálculo

$$T(m, p) = O(m) + O(1) + O(p \cdot m) + O(1)$$

- Como $O(p \cdot m)$ domina:

$$T(m, p) = O(p \cdot m)$$

Caso promedio realista

- Como ahora se busca la firma más grave, no se corta en la primera coincidencia. Por tanto, en promedio también se revisan todos los patrones:

$$O(p \cdot m)$$

Tiempo promedio

$O(p \cdot m)$

Memoria

$O(m)$

2.3 firmas.py

Este archivo gestiona la base de firmas mediante diccionarios.

BaseFirmas.__init__()

Operación

- Crea dos diccionarios vacíos:
 - **firmas_por_hash**
 - **firmas_por_patron**

Cálculo

$T(n) = c$

Tiempo promedio

$O(1)$

Memoria

$O(1)$

- Inicialmente vacía, pero crecerá a:

$O(p + h)$

agregar_firma_hash()

Operación

- Inserta una firma en el diccionario de hashes.

Cálculo promedio

- Las tablas hash permiten inserción promedio constante:

$$T(h) = O(1)$$

Peor caso teórico

- Si hubiera muchas colisiones:

$$O(h)$$

Tiempo promedio

$$O(1)$$

Memoria

$$O(1)$$

- por firma insertada.

agregar_firma_patron()

Operación

- Inserta una firma en el diccionario de patrones.

Tiempo promedio

$$O(1)$$

Memoria

$O(1)$

- por patrón añadido.

buscar_por_hash()

Operación

- Busca si el hash de un archivo existe en la base.

Cálculo promedio

$T(h) = O(1)$

- porque el acceso a diccionarios en Python es promedio constante.

Peor caso teórico

$O(h)$

- si se produjeran colisiones masivas.

Tiempo promedio

$O(1)$

Memoria

$O(1)$

buscar_patron_en_contenido()

Operación

- Busca todos los patrones dentro del contenido y devuelve el patrón de mayor severidad.

Cálculo

- Para cada patrón, se comprueba si aparece dentro del contenido:

```
for patron in patrones:
    if patron in contenido:
```

- Si hay p patrones y el contenido tiene tamaño m :

$$T(p, m) = p \cdot O(m)$$

- Por tanto:

$$T(p, m) = O(p \cdot m)$$

- Antes se podía devolver el primer patrón encontrado. Ahora, como se busca el de mayor severidad, el sistema debe revisar todos los patrones aunque ya haya encontrado uno.
- Por tanto, incluso en caso promedio:

$$O(p \cdot m)$$

Tiempo promedio

$$O(p \cdot m)$$

Memoria

$$O(1)$$

- solo guarda una variable temporal `firma_mas_grave`.

`cantidad_firmas()`

Operación

- Suma la cantidad de hashes y patrones.

Cálculo

$$\text{len}(\text{diccionario})$$

- es constante en Python.

$T(s) = O(1)$

Tiempo promedio

$O(1)$

Memoria

$O(1)$

obtener_hashes_registrados()

Operación

- Devuelve el diccionario de hashes.

Tiempo promedio

$O(1)$

- porque devuelve una referencia, no copia el diccionario.

Memoria

$O(1)$

2.4 main.py

Este archivo ejecuta el sistema por consola.

ejecutar_demo()

Operación

- Crea sistema, base de firmas, motor y ejecuta escaneo.

Cálculo

- Escaneo rápido:

$$O(n + f \cdot m)$$

- Escaneo profundo:

$$O(n + f \cdot p \cdot m)$$

- Después imprime resultados:

$$O(r)$$

Tiempo promedio

$$O(n + f \cdot p \cdot m)$$

- si se considera la ejecución profunda.
-

2.5 modelo.py

Este archivo define las clases base del sistema: archivos, carpetas, firmas y resultados.

NodoSistema.obtener_ruta()

Operación

- Construye la ruta lógica de un archivo o carpeta.

Cálculo

$$T(1) = O(1)$$

- Donde **1** es la longitud de la ruta generada.
- Aunque normalmente se aproxima a **O(1)**, si se analiza de forma estricta, concatenar cadenas depende de su longitud.

Tiempo promedio

$$O(1)$$

Memoria

$O(1)$

- porque se crea una nueva cadena con la ruta.

`Archivo.calcular_hash()`

Operación

- Calcula el hash SHA-256 del contenido del archivo.

Cálculo

- El algoritmo debe leer todo el contenido:

$$T(m) = c1 \cdot m + c2$$

- Por tanto:

$$T(m) \in O(m)$$

Tiempo promedio

$O(m)$

Memoria

$O(m)$

- porque se genera una representación en bytes del contenido con `encode()`.

`Archivo.tamano()`

Operación

- Devuelve la longitud del contenido.

Cálculo

- En Python, `len()` sobre una cadena es constante:

$$T(m) = c$$

Tiempo promedio

$$O(1)$$

Memoria

$$O(1)$$

Carpeta.agregar_hijo()

Operación

- Añade un archivo o carpeta a la lista de hijos.

Cálculo

- Python usa listas dinámicas. El coste promedio de **append()** es:

$$O(1)$$

- Aunque en un redimensionamiento puntual podría ser:

$$O(k)$$

- donde **k** es el número de elementos actuales.

Tiempo promedio

$$O(1)$$

Memoria

$$O(1)$$

- por cada nueva referencia añadida.
-

2.6 motor.py

Este archivo contiene el núcleo algorítmico del sistema.

MotorEscaneo.__init__()

Operación

- Guarda referencias a:
 - base de firmas,
 - estrategia,
 - lista de resultados.

Tiempo promedio

$O(1)$

Memoria

$O(1)$

- La lista de resultados crecerá después a:

$O(f)$

MotorEscaneo.escanear()

Operación

- Limpia resultados e inicia el recorrido recursivo.

```
self.resultados = []  
self._escanear_nodo(raiz, "")
```

Cálculo

- El coste depende de `_escanear_nodo()`.
- Con escaneo rápido:

$T(n, f, m) = O(n) + O(f \cdot m)$

- Resultado:

$$O(n + f \cdot m)$$

- Con escaneo profundo:

$$T(n, f, p, m) = O(n) + O(f \cdot p \cdot m)$$

- Resultado:

$$O(n + f \cdot p \cdot m)$$

_escanear_nodo()

Operación

- Recorre recursivamente el árbol de archivos.

Recurrencia

- En un árbol general:

$$T(n) = T(n_1) + T(n_2) + \dots + T(n_k) + c$$

- Donde:

$$n_1 + n_2 + \dots + n_k = n - 1$$

- Como cada nodo se visita una vez:

$$T(n) = O(n)$$

Resolución simplificada

- En el peor caso degenerado:

$$T(n) = T(n - 1) + c$$

- Expansión:

$T(n) = T(n-1) + c$
 $T(n) = T(n-2) + 2c$
 $T(n) = T(n-3) + 3c$
...
 $T(n) = T(1) + (n-1)c$
 $T(n) = n \cdot c$
 $T(n) \in O(n)$

Tiempo promedio

$O(n)$

Pila

- La profundidad de la pila depende de d :

$O(d)$

- Si el árbol está muy desequilibrado:

$O(n)$

- Si está equilibrado:

$O(\log n)$

generar_resumen()

Operación

- Cuenta cuántos resultados son:
 - limpios,
 - sospechosos,
 - maliciosos.

Cálculo

- Hay tres recorridos sobre r resultados:

$$T(r) = r + r + r$$

$$T(r) = 3r$$

$$T(r) \in O(r)$$

Tiempo promedio

$$O(r)$$

- Como normalmente $r = f$:

$$O(f)$$

Memoria

$$O(1)$$

3. Complejidad mejor, promedio y peor caso

3.1 app.py

Este archivo contiene la interfaz gráfica y coordina el sistema.

crear_sistema_simulado()

```
return Carpeta("root")
```

$T(n) = O(1)$
Memoria: $O(1)$

crear_base_firmas()

Carga las firmas iniciales. Si hay s firmas:

$T(s) = O(s)$

Como ahora hay 12 firmas fijas:

Mejor: $O(1)$
Promedio: $O(1)$
Peor: $O(1)$

Pero generalizado:

$O(s)$

Memoria: $O(s)$
Pila: $O(1)$

cargar_archivos_guardados()

Carga archivos desde JSON.

$$T(f, m) = O(f \cdot m)$$

Porque lee **f** archivos guardados con contenido medio **m**.

Mejor caso: $O(1)$, si no existe JSON
Promedio: $O(f \cdot m)$
Peor caso: $O(f \cdot m)$
Memoria: $O(f \cdot m)$
Pila: $O(1)$

guardar_archivos()

Guarda los archivos en JSON.

$$T(f, m) = O(f \cdot m)$$

Mejor: $O(1)$, si no hay archivos
Promedio: $O(f \cdot m)$
Peor: $O(f \cdot m)$
Memoria: $O(f \cdot m)$

obtener_o_crear_carpeta_usuario()

Busca la carpeta de usuario.

$$T(c) = O(c)$$

Mejor: $O(1)$, si la encuentra al principio
Promedio: $O(c)$
Peor: $O(c)$
Memoria: $O(1)$

crear_interfaz()

Crea widgets fijos.

$T(n) = O(1)$
Memoria: $O(1)$

actualizar_tarjetas()

Cuenta resultados.

$T(r) = O(r)$

Mejor: $O(1)$, sin resultados
Promedio: $O(r)$
Peor: $O(r)$
Memoria: $O(1)$

obtener_archivos_anadidos()

Recorre carpetas y archivos.

$T(n) = O(n)$

Mejor: $O(1)$, si no hay archivos
Promedio: $O(n)$
Peor: $O(n)$
Memoria: $O(f)$, porque devuelve una lista de archivos

anadir_archivo()

Operaciones:

leer archivo: $O(m)$
buscar carpeta: $O(c)$
añadir a lista: $O(1)$
guardar JSON: $O(f \cdot m)$

Total:

```
T(f, m, c) = O(m) + O(c) + O(1) + O(f · m)
T(f, m, c) ∈ O(f · m)
```

```
Mejor caso: O(1), si el usuario cancela selección
Promedio: O(f · m)
Peor caso: O(f · m)
Memoria: O(m) + O(f · m)
Pila: O(1)
```

eliminar_archivo()

Operaciones:

```
obtener archivos: O(n)
eliminar seleccionado: O(f)
guardar JSON: O(f · m)
```

Total:

```
T(n, f, m) = O(n + f + f · m)
T(n, f, m) ∈ O(n + f · m)
```

```
Mejor: O(1), si no hay archivos
Promedio: O(n + f · m)
Peor: O(n + f · m)
Memoria: O(f)
```

ejecutar_escaneo()

Operaciones:

```
obtener archivos: O(n)
motor.escanear(): depende de estrategia
mostrar resultados: O(r)
registrar hashes: hasta O(f · m)
```

Con escaneo rápido:

$$T = O(n + f \cdot m)$$

Con escaneo profundo:

$$T = O(n + f \cdot p \cdot m)$$

Mejor: $O(1)$, si no hay archivos
Promedio rápido: $O(n + f \cdot m)$
Promedio profundo: $O(n + f \cdot p \cdot m)$
Peor rápido: $O(n + f \cdot m)$
Peor profundo: $O(n + f \cdot p \cdot m)$
Memoria: $O(f + d)$
Pila: $O(d)$

registrar_hash_detectado()

Busca el archivo detectado y registra su hash.

buscar archivo: $O(f)$
calcular hash: $O(m)$
insertar hash: $O(1)$

Total:

$$O(f + m)$$

Mejor: $O(m)$, si está al principio
Promedio: $O(f + m)$
Peor: $O(f + m)$
Memoria: $O(m)$
Pila: $O(1)$

mostrar_archivos()

Calcula hash de todos los archivos.

$$T(f, m) = O(f \cdot m)$$

Mejor: $O(1)$, si no hay archivos

Promedio: $O(f \cdot m)$

Peor: $O(f \cdot m)$

Memoria: $O(f)$

mostrar_hashes()

Recorre hashes.

$$T(h) = O(h)$$

Mejor: $O(1)$

Promedio: $O(h)$

Peor: $O(h)$

Memoria: $O(h)$

mostrar_firmas()

Recorre patrones y hashes.

$$T(p, h) = O(p + h)$$

Mejor: $O(1)$

Promedio: $O(p + h)$

Peor: $O(p + h)$

Memoria: $O(p + h)$

exportar_informe_html()

Recorre resultados y hashes.

$$T(r, h) = O(r + h)$$

Como $r \approx f$:

$$O(f + h)$$

Mejor: $O(1)$, si no hay resultados
Promedio: $O(f + h)$
Peor: $O(f + h)$
Memoria: $O(f + h)$

limpiar_tabla()

Borra filas.

$$T(r) = O(r)$$

limpiar_resultados()

Limpia tabla y actualiza tarjetas.

$$T(r, f) = O(r + f)$$

borrar_archivos_guardados()

Borra todos los archivos persistentes.

$$T(r, f) = O(r + f)$$

3.2 estrategias.py

Este archivo implementa el polimorfismo mediante distintas estrategias de escaneo.

Clase abstracta **EstrategiaEscaneo**

```
class EstrategiaEscaneo(ABC):  
    @abstractmethod  
    def escanear_archivo(...):  
        pass
```

No ejecuta análisis.

Tiempo: $O(1)$
Memoria: $O(1)$
Recurrencia: no

EscaneoRapido.escanear_archivo()

```
hash_archivo = archivo.calcular_hash()  
firma = base_firmas.buscar_por_hash(hash_archivo)
```

Operaciones

1. Calcula hash: $O(m)$
2. Busca hash: $O(1)$ promedio
3. Crea resultado: $O(1)$

Cálculo

$$T(m, h) = O(m) + O(1) + O(1)$$
$$T(m, h) = O(m)$$

Casos

Mejor caso: $O(m)$
Promedio: $O(m)$
Peor caso: $O(m)$

- El hash siempre necesita leer todo el archivo.

Resolución

```
T(m) = c · m + k  
T(m) ∈ O(m)
```

```
Fase de bajada: no aplica  
Fase de subida: no aplica  
Profundidad de pila: O(1)  
Memoria adicional: O(m)
```

EscaneoProfundo.escanear_archivo()

```
hash_archivo = archivo.calcular_hash()  
firma_hash = base_firmas.buscar_por_hash(hash_archivo)
```

Primero calcula hash.

```
O(m) + O(1)
```

Después, si no encuentra hash:

```
firma_patron = base_firmas.buscar_patron_en_contenido(archivo.contenido)
```

Coste:

```
O(p · m)
```

Cálculo completo

```
T(m, p) = O(m) + O(1) + O(p · m) + O(1)  
T(m, p) = O(p · m)
```

Casos

```
Mejor caso: O(m), si el hash ya está registrado  
Promedio: O(p · m), si no hay hash y se analizan patrones
```

Peor caso: $O(p \cdot m)$, si no hay hash y se revisan todos los patrones

Recurrencia asociada

- La parte de patrones equivale a:

$$T(p) = T(p - 1) + O(m)$$

- Expansión:

$$\begin{aligned} T(p) &= T(p - 1) + m \\ T(p) &= T(p - 2) + 2m \\ &\dots \\ T(p) &= T(0) + p \cdot m \\ T(p) &\in O(p \cdot m) \end{aligned}$$

Fase de bajada: revisión de patrones

Fase de subida: no aplica

Profundidad máxima de pila: $O(1)$

Memoria adicional: $O(m)$

3.3 firmas.py

Este archivo gestiona la base de firmas mediante diccionarios.

Tipo de complejidad

Principalmente usa **tablas hash**, por lo que muchas operaciones son **$O(1)$** promedio. La operación más costosa es buscar patrones dentro del contenido.

Constructor `__init__()`

```
self.firmas_por_hash = {}  
self.firmas_por_patron = {}
```

Crea dos diccionarios.

```
T(n) = c  
O asintótico:  $O(1)$ 
```

Mejor caso: $O(1)$
Promedio: $O(1)$
Peor caso: $O(1)$
Memoria inicial: $O(1)$
Memoria final: $O(h + p)$

agregar_firma_hash()

```
self.firmas_por_hash[hash_archivo] = firma
```

Inserta una firma por hash.

Tipo de complejidad: constante promedio
 $T(h) = O(1)$

Mejor caso: $O(1)$
Promedio: $O(1)$
Peor caso teórico: $O(h)$, por colisiones
Memoria: $O(1)$ por firma
Pila: $O(1)$

agregar_firma_patron()

```
self.firmas_por_patron[patron] = firma
```

Inserta una firma por patrón.

Mejor caso: $O(1)$
Promedio: $O(1)$
Peor caso teórico: $O(p)$
Memoria: $O(1)$
Pila: $O(1)$

buscar_por_hash()

```
return self.firmas_por_hash.get(hash_archivo)
```

Busca un hash en un diccionario.

Tipo de complejidad: constante promedio
Recurrencia: no
 $T(h) = O(1)$

Mejor caso: $O(1)$
Promedio: $O(1)$
Peor caso teórico: $O(h)$
Memoria: $O(1)$
Pila: $O(1)$

buscar_patron_en_contenido()

```
firma_mas_grave = None

for patron, firma in self.firmas_por_patron.items():
    if patron in contenido:
        if firma_mas_grave is None or firma.severidad > firma_mas_grave.severidad:
            firma_mas_grave = firma

return firma_mas_grave
```

Esta función recorre todos los patrones y devuelve la firma de mayor severidad.

Tipo de complejidad

- Es una complejidad **lineal multiplicada**, porque por cada patrón puede recorrer el contenido.

$$T(p, m) = p \cdot O(m)$$
$$T(p, m) = O(p \cdot m)$$

Recurrencia equivalente

- Aunque está implementado con un bucle, se puede expresar como:

$$T(p) = T(p - 1) + O(m)$$
$$T(0) = O(1)$$

Resolución por expansión

```
T(p) = T(p - 1) + m
T(p) = T(p - 2) + 2m
T(p) = T(p - 3) + 3m
...
T(p) = T(0) + p · m
T(p) = 1 + p · m
T(p) ∈ O(p · m)
```

Casos

```
Mejor caso: O(p · m)
Promedio: O(p · m)
Peor caso: O(p · m)
```

- Antes podría cortarse en la primera coincidencia, pero ahora debe revisar todos los patrones para quedarse con el de mayor severidad. Por eso el mejor, promedio y peor caso quedan igual en cuanto al número de patrones revisados.

```
Fase de bajada: se reduce el número de patrones pendientes p → p-1 → ... → 0
Fase de subida: no hay subida real, porque no es recursivo
Profundidad de pila: O(1)
Memoria adicional: O(1)
```

cantidad_firmas()

```
return len(self.firmas_por_hash) + len(self.firmas_por_patron)
```

```
T(s) = O(1)
Mejor: O(1)
Promedio: O(1)
Peor: O(1)
Memoria: O(1)
```

obtener_hashes_registrados()

```
return self.firmas_por_hash
```

Devuelve una referencia al diccionario.

```
T(h) = O(1)
Memoria: O(1)
```

3.4 main.py

Este archivo permite ejecutar el sistema por consola.

`ejecutar_demo()`

```
sistema = crear_sistema_simulado()
base = crear_base_firmas()
motor = MotorEscaneo(base, estrategia)
resultados = motor.escanear(sistema)
```

Depende del motor. Escaneo rápido:

```
O(n + f · m)
```

Escaneo profundo:

```
O(n + f · p · m)
```

Después imprime resultados:

```
O(r)
```

Casos

```
Mejor: O(1), si no hay archivos
Promedio rápido: O(n + f · m)
Promedio profundo: O(n + f · p · m)
Peor rápido: O(n + f · m)
Peor profundo: O(n + f · p · m)
```

Recurrencia La recursividad no está en **main.py**, pero llama al motor:

$$T(n) = \sum T(n_i) + c$$

Pila

$$O(d)$$

Memoria

$$O(n + f + p + h + d)$$

3.5 modelo.py

Este archivo define las clases principales del modelo de datos: **NodoSistema**, **Archivo**, **Carpeta**, **FirmaMalware** y **ResultadoEscaneo**.

Tipo de complejidad

- Principalmente es **constante**, salvo el cálculo del hash, que depende del tamaño del archivo.

Clase **NodoSistema**

```
@dataclass
class NodoSistema:
    nombre: str
```

Crea la estructura base de un nodo.

```
Tiempo: O(1)
Memoria: O(1)
```

Método **obtener_ruta()**

```
def obtener_ruta(self, ruta_padre: str = "") -> str:
    if ruta_padre:
        return f"{ruta_padre}/{self.nombre}"
    return self.nombre
```

Este método genera la ruta lógica de un archivo o carpeta.

Tipo de complejidad: lineal respecto a la longitud de la ruta
Recurrencia: no tiene recursividad
 $T(1) = O(1)$

Donde **1** es la longitud de la cadena generada.

Mejor caso: $O(1)$, si no hay ruta padre
Promedio: $O(1)$
Peor caso: $O(1)$, si la ruta es larga
Memoria: $O(1)$
Pila: $O(1)$

No hay fase de bajada ni subida porque no es recursiva.

Clase **Archivo**

```
@dataclass
class Archivo(NodoSistema):
    contenido: str
    extension: str = ".txt"
```

Guarda el contenido de un archivo.

Tiempo de creación: $O(1)$
Memoria: $O(m)$

La memoria depende del tamaño del contenido.

Método **calcular_hash()**

```
def calcular_hash(self) -> str:
    return hashlib.sha256(self.contenido.encode("utf-8")).hexdigest()
```

Calcula el hash SHA-256 del contenido.

Tipo de complejidad: lineal
Recurrencia: no recursiva
 $T(m) = c \cdot m + k$

Resolución:

```
T(m) = c · m + k  
T(m) ∈ O(m)
```

```
Mejor caso: O(m)  
Promedio: O(m)  
Peor caso: O(m)
```

Aunque el archivo sea pequeño o grande, debe leerse completo para calcular el hash.

```
Fase de bajada: no aplica  
Fase de subida: no aplica  
Profundidad máxima de pila: O(1)  
Memoria adicional: O(m)
```

La memoria adicional aparece por `encode("utf-8")`, que crea una representación en bytes del contenido.

Método `tamano()`

```
def tamano(self) -> int:  
    return len(self.contenido)
```

Devuelve la longitud del contenido.

```
Tipo de complejidad: constante  
T(m) = c  
O asintótico: O(1)
```

```
Mejor caso: O(1)  
Promedio: O(1)  
Peor caso: O(1)  
Memoria: O(1)  
Pila: O(1)
```

En Python, `len()` sobre cadenas es constante.

Clase **Carpeta**

```
@dataclass
class Carpeta(NodoSistema):
    hijos: List[NodoSistema] = field(default_factory=list)
```

Representa una carpeta que contiene hijos.

Tiempo de creación: $O(1)$
Memoria: $O(k)$

Donde **k** es el número de hijos.

Método **agregar_hijo()**

```
def agregar_hijo(self, nodo: NodoSistema) -> None:
    self.hijos.append(nodo)
```

Añade un archivo o carpeta a la lista.

Tipo de complejidad: constante amortizada
 $T(k) = O(1)$

Mejor caso: $O(1)$
Promedio: $O(1)$
Peor caso puntual: $O(k)$, si la lista se redimensiona
Peor caso amortizado: $O(1)$
Memoria: $O(1)$
Pila: $O(1)$

No tiene recursividad.

Clases **FirmaMalware** y **ResultadoEscaneo**

Solo almacenan información.

```
Tiempo de creación: O(1)
Memoria: O(1) + tamaño de cadenas
Recurrencia: no
Pila: O(1)
```

3.6 motor.py

Este archivo contiene el recorrido recursivo principal.

`__init__()`

```
self.base_firmas = base_firmas
self.estrategia = estrategia
self.resultados = []
```

```
Tiempo: O(1)
Memoria inicial: O(1)
Memoria final: O(f)
```

`escanear()`

```
self.resultados = []
self._escanear_nodo(raiz, "")
return self.resultados
```

Llama al recorrido recursivo. Con escaneo rápido:

```
O(n + f · m)
```

Con escaneo profundo:

```
O(n + f · p · m)
```

`_escanear_nodo()`

```
ruta_actual = nodo.obtener_ruta(ruta_padre)

if isinstance(nodo, Archivo):
    resultado = self.estrategia.escanear_archivo(...)
    self.resultados.append(resultado)

elif isinstance(nodo, Carpeta):
    for hijo in nodo.hijos:
        self._escanear_nodo(hijo, ruta_actual)
```

Tipo de complejidad

- Es **recursiva lineal sobre un árbol**.

Recurrancia general

$$T(n) = T(n_1) + T(n_2) + \dots + T(n_k) + c$$

- Donde:

$$n_1 + n_2 + \dots + n_k = n - 1$$

- Cada nodo se visita una vez.

$$T(n) \in O(n)$$

Recurrancia simplificada

- En el peor caso, si el árbol está degenerado:

$$T(n) = T(n - 1) + c$$

Resolución por expansión

$$\begin{aligned} T(n) &= T(n - 1) + c \\ T(n) &= T(n - 2) + 2c \\ T(n) &= T(n - 3) + 3c \\ &\dots \\ T(n) &= T(1) + (n - 1)c \end{aligned}$$

$T(n) = n \cdot c$
 $T(n) \in O(n)$

Fase de bajada

- La fase de bajada ocurre cuando el algoritmo entra en carpetas:

root → carpeta → subcarpeta → archivo

- Cada llamada procesa un nodo y baja hacia sus hijos.

Fase de subida

- La fase de subida ocurre cuando termina una rama y vuelve al nodo anterior:

archivo → subcarpeta → carpeta → root

- Después continúa con otros hijos.

Casos

Mejor caso: $O(1)$, si solo existe la raíz vacía
Promedio: $O(n)$
Peor caso: $O(n)$

Profundidad máxima de pila

$O(d)$

- Donde **d** es la profundidad máxima del árbol.

Si el árbol es degenerado: $O(n)$
Si el árbol es equilibrado: $O(\log n)$

Memoria

Resultados: $O(f)$
Pila recursiva: $O(d)$
Memoria total auxiliar: $O(f + d)$

generar_resumen()

```
total = len(self.resultados)
limpios = sum(...)
sospechosos = sum(...)
maliciosos = sum(...)
```

Recorre la lista de resultados varias veces.

```
T(f) = f + f + f
T(f) = 3f
T(f) ∈ O(f)
```

```
Mejor caso: O(f)
Promedio: O(f)
Peor caso: O(f)
Memoria: O(1)
Pila: O(1)
```

4. Conclusión: propuestas a futuro y optimización

MalScan ha permitido desarrollar un prototipo funcional de escáner de malware por firmas, aplicando estructuras de datos, programación orientada a objetos, polimorfismo y recursividad dentro de un contexto realista de ciberseguridad. El sistema permite añadir archivos, analizarlos mediante hashes y patrones, clasificarlos según su nivel de riesgo, visualizar las firmas cargadas, exportar informes y conservar los archivos añadidos entre ejecuciones.

Como propuestas a futuro, el sistema podría mejorarse incorporando una base de datos persistente para guardar firmas, hashes, resultados históricos e informes generados. También sería interesante permitir la carga automática de firmas desde archivos externos o repositorios actualizados, simulando el funcionamiento de las actualizaciones de un antivirus real.

Desde el punto de vista funcional, MalScan podría incorporar un sistema de cuarentena para aislar archivos maliciosos, un módulo de monitorización en tiempo real de carpetas y una opción para programar escaneos automáticos. Además, se podrían añadir estadísticas avanzadas con gráficos sobre amenazas detectadas, tipos de malware, niveles de riesgo y evolución de los análisis.

En cuanto a la optimización algorítmica, la mejora más importante sería sustituir la búsqueda lineal de patrones por una estructura más eficiente. Actualmente, el escaneo profundo tiene un coste aproximado de $O(p \cdot m)$, donde p es el número de patrones y m el tamaño del archivo. Con una estructura optimizada, el sistema podría analizar múltiples patrones de forma más eficiente, especialmente si la base de firmas crece.

También se podría optimizar el cálculo de hashes evitando recalcular el hash de archivos que no han cambiado. Para ello, el sistema podría almacenar el hash anterior junto con la fecha de modificación del archivo. Si el archivo no se ha modificado, MalScan podría reutilizar el resultado anterior, reduciendo el tiempo de análisis.

Aunque se trata de un prototipo educativo, su arquitectura modular permite ampliarlo fácilmente y convertirlo en una herramienta más completa, eficiente y cercana al funcionamiento de un antivirus real.